Workshop

Image Processing with Point Processing Techniques



Image Processing with Point Processing Techniques

The objective of this task is to apply and understand different **point processing techniques** in image processing. You will use Python and OpenCV to manipulate a grayscale image using the following methods:

1. **Brightness Adjustment** – Modify the overall brightness of an image by adding a fixed value to all pixel intensities.
2. **Contrast Adjustment** – Scale pixel intensities to enhance the difference between dark and bright regions.
3. **Gamma Correction** – Apply a non-linear transformation to adjust brightness and contrast dynamically.
4. **Histogram Equalization** – Improve contrast by redistributing pixel intensities to spread them more evenly.
5. **Local Table Transformations (CLAHE - Contrast Limited Adaptive Histogram Equalization)** – Apply histogram equalization locally to enhance contrast in different regions.
6. **Negative Transformation** – Invert pixel values to create a negative effect of the image.

By implementing these techniques, you will analyse how they impact image brightness, contrast, and histogram distributions, and compare their effectiveness for different types of images.

Python code

stance_measurment.py | Connectivity_demo.py | DistanceinMM.py | DistanceMM_CV.py | PointProcessing.py | PointProcessing2.py

C: > Users > jdehm > OneDrive > Documents > Presentation > Kingston > ComputerVisionCourse2025 > DigitalImageProcessing2025 > Workshop1 > PointProcessing2.py

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def plot_histogram(image, title="Histogram"):
    hist, bins = np.histogram(image.flatten(), 256, [0, 256])
    plt.figure()
    plt.title(title)
    plt.xlabel("Pixel Value")
    plt.ylabel("Frequency")
    plt.plot(hist)
    plt.xlim([0, 256])
    plt.show()

def adjust_brightness(image, value):
    """Increase brightness by adding a fixed value."""
    return cv2.convertScaleAbs(image, alpha=1, beta=value)

def adjust_contrast(image, factor):
    """Enhance contrast by scaling pixel values."""
    return cv2.convertScaleAbs(image, alpha=factor, beta=0)

def gamma_correction(image, gamma):
    """Apply gamma correction (non-linear transformation)."""
    invGamma = 1.0 / gamma
    table = np.array([(i / 255.0) ** invGamma * 255 for i in range(256)]).astype("uint8")
    return cv2.LUT(image, table)

def histogram_equalization(image):
    """Enhance contrast using histogram equalization."""
    return cv2.equalizeHist(image)

def local_table_transformation(image, clip_limit=2.0, tile_grid_size=(8,8)):
    """Adaptive Histogram Equalization with CLAHE (local contrast enhancement)."""
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)
    return clahe.apply(image)

```

```python

def negative_transformation(image):
    """Invert pixel intensities (negative effect)."""
    return 255 - image

def main():
    # Load the uploaded grayscale image
    image_path = "images/peper.tiff"  # Replace with the actual path
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        print("Error: Could not read the image file.")
        return

    # Display the original image and its histogram
    cv2.imshow("Original Image", image)
    plot_histogram(image, "Original Image Histogram")

    # Apply processing techniques
    bright_image = adjust_brightness(image, 50)
    contrast_image = adjust_contrast(image, 1.5)
    gamma_image = gamma_correction(image, 2.0)
    hist_eq_image = histogram_equalization(image)
    clahe_image = local_table_transformation(image)
    negative_image = negative_transformation(image)

    # Display results
    cv2.imshow("Brightened Image", bright_image)
    plot_histogram(bright_image, "Brightened Image Histogram")

    cv2.imshow("Contrast Adjusted Image", contrast_image)
    plot_histogram(contrast_image, "Contrast Adjusted Histogram")

    cv2.imshow("Gamma Corrected Image", gamma_image)
    plot_histogram(gamma_image, "Gamma Corrected Histogram")

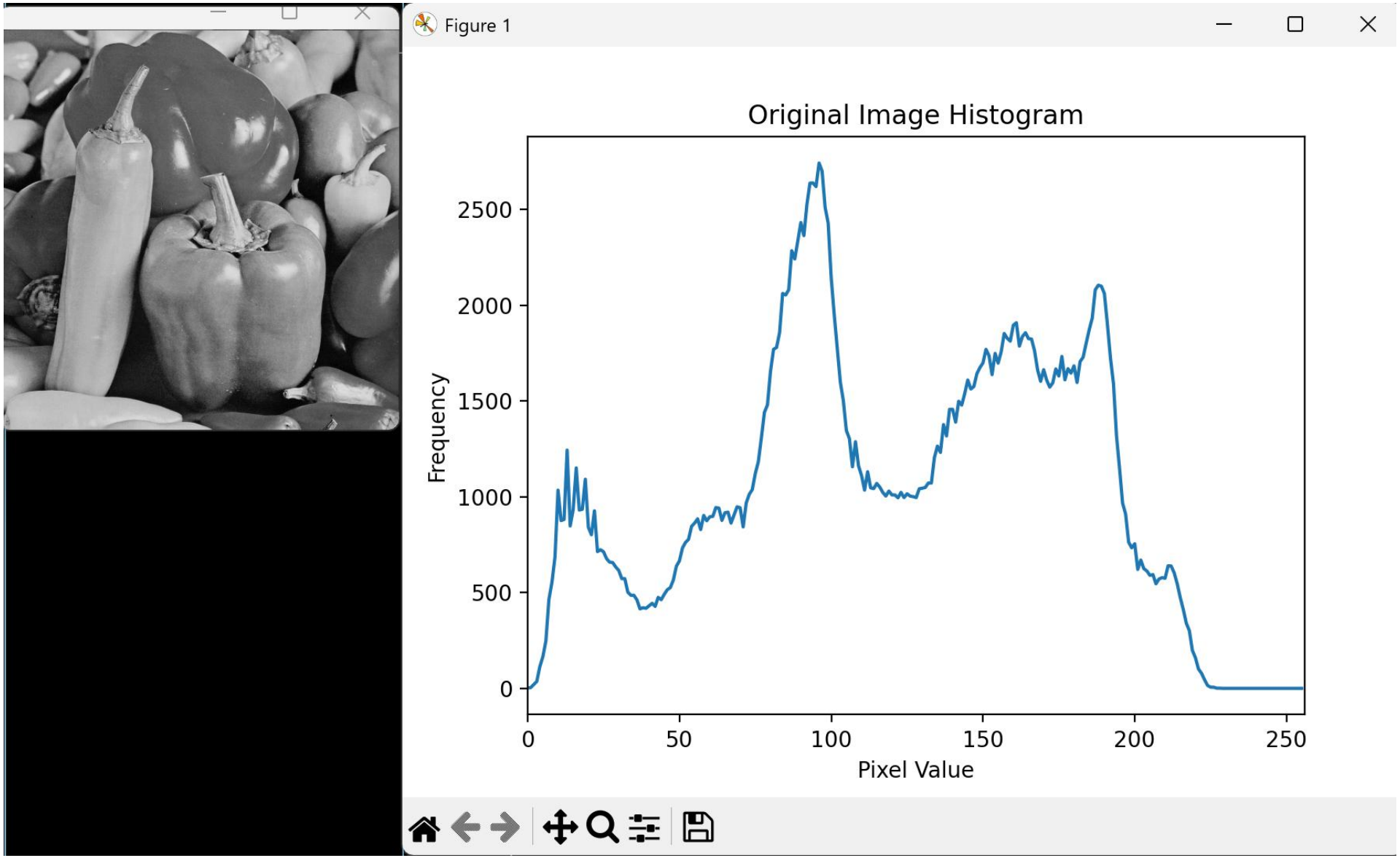
    cv2.imshow("Histogram Equalized Image", hist_eq_image)
    plot_histogram(hist_eq_image, "Histogram Equalization Histogram")

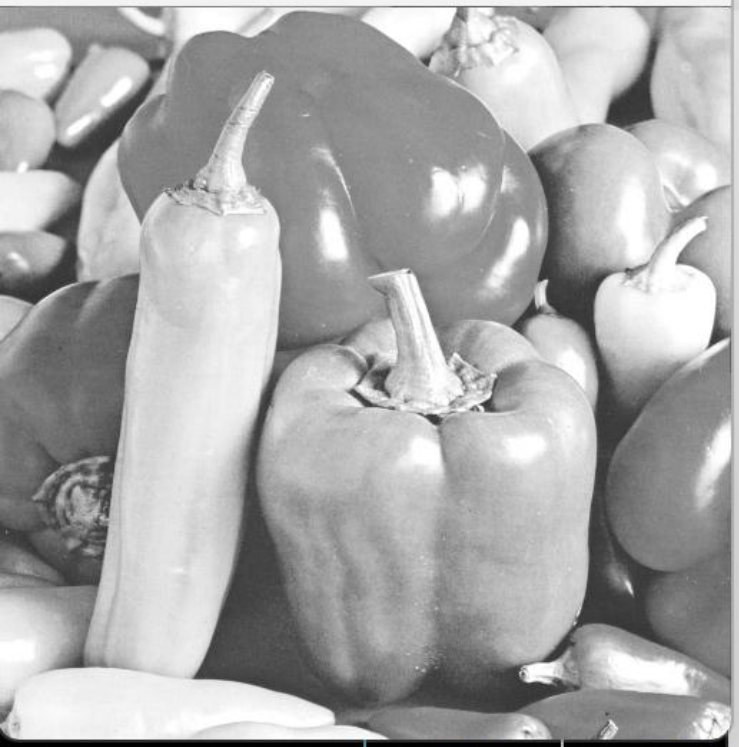
    cv2.imshow("CLAHE Image", clahe_image)
    plot_histogram(clahe_image, "CLAHE Histogram")

    cv2.imshow("Negative Image", negative_image)
    plot_histogram(negative_image, "Negative Image Histogram")

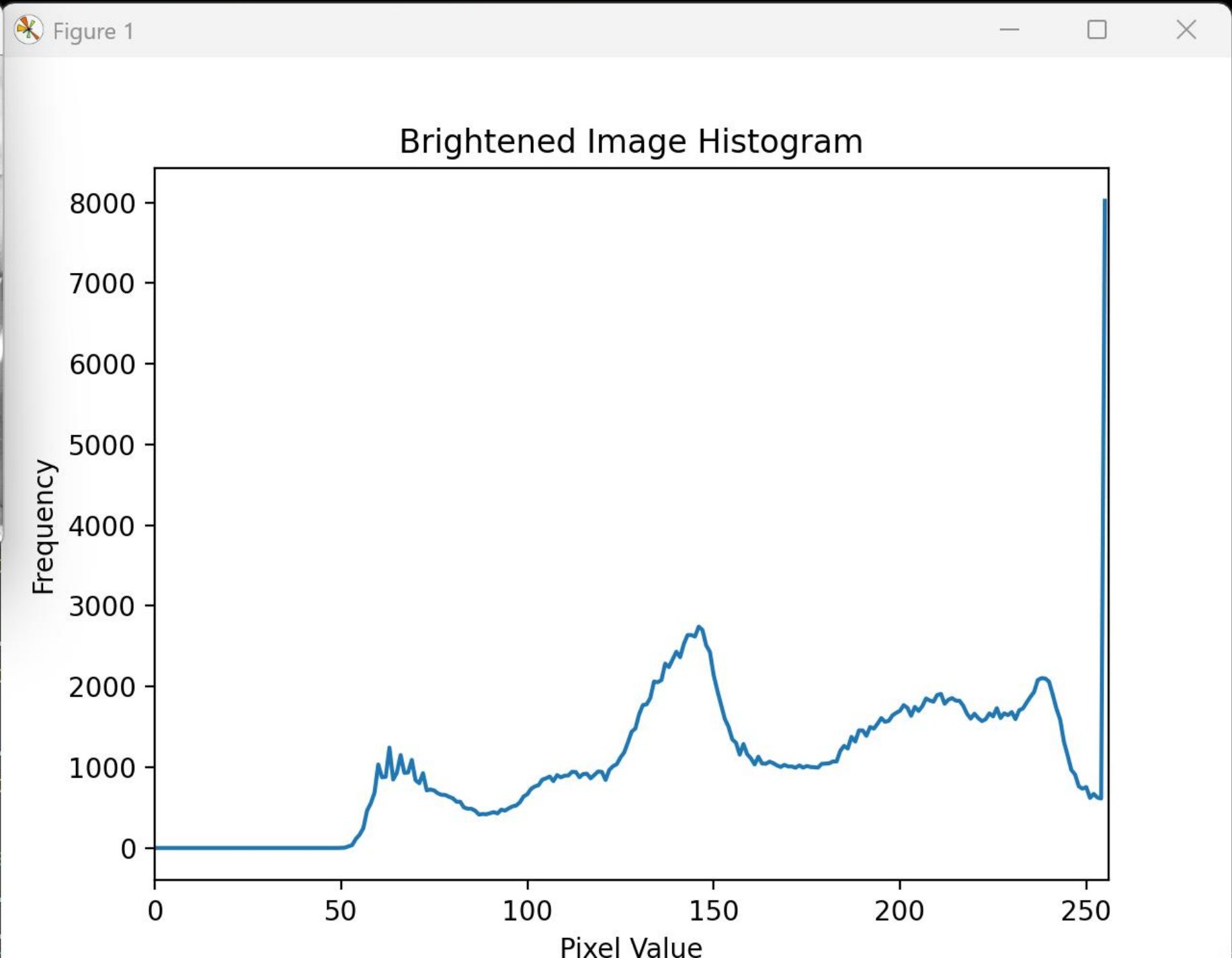
    # Wait and close windows
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```


Figure 1
Original Image Histogram
Frequency
2500
2000
1500
1000
500
0
0
50
100
150
200
250
Pixel Value




```
73      plot
74
75      cv2.
76      plot
77
78      cv2.
79      plot
80
81      # Wa
82      cv2.
83      cv2.
```



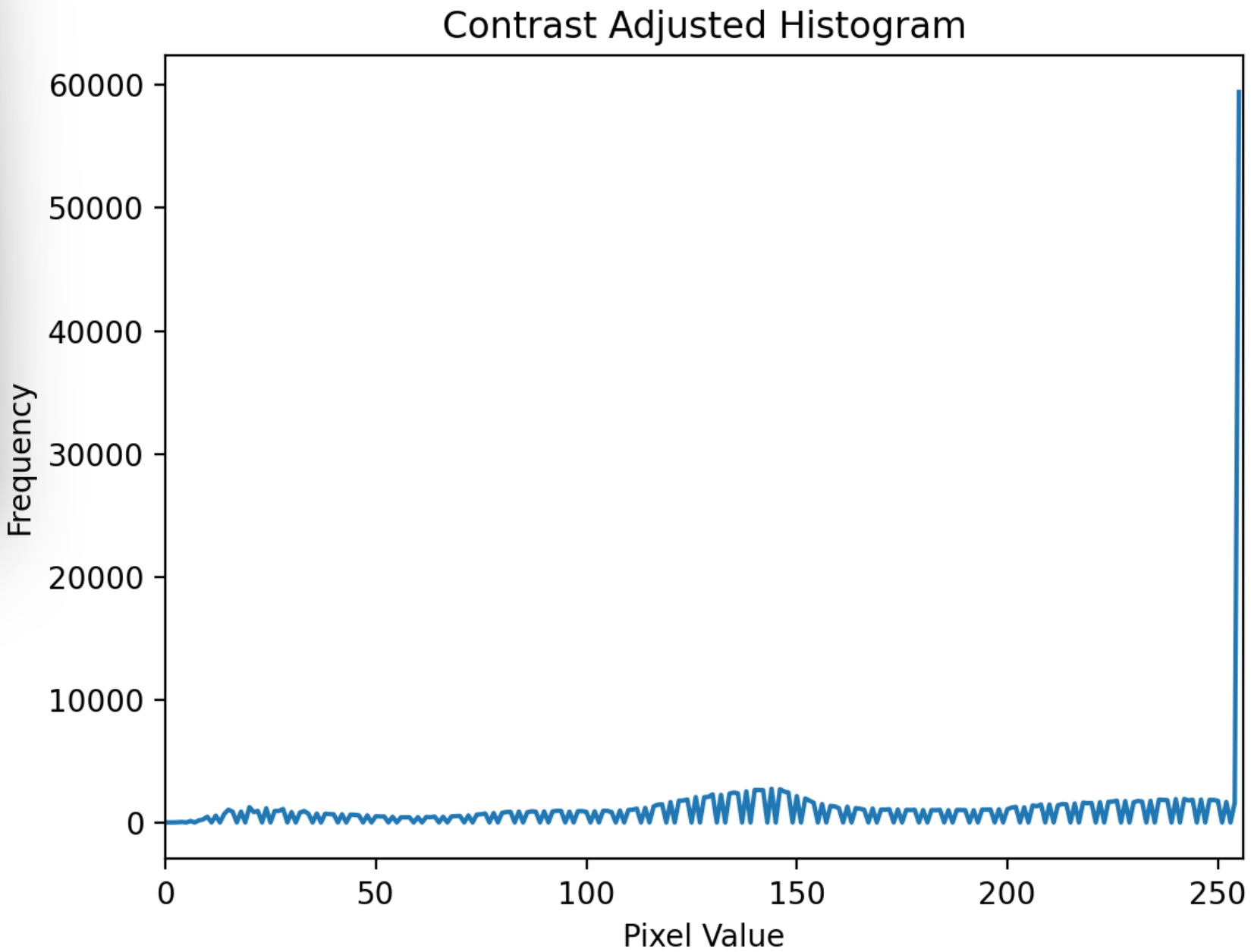
Contrast Adjus...



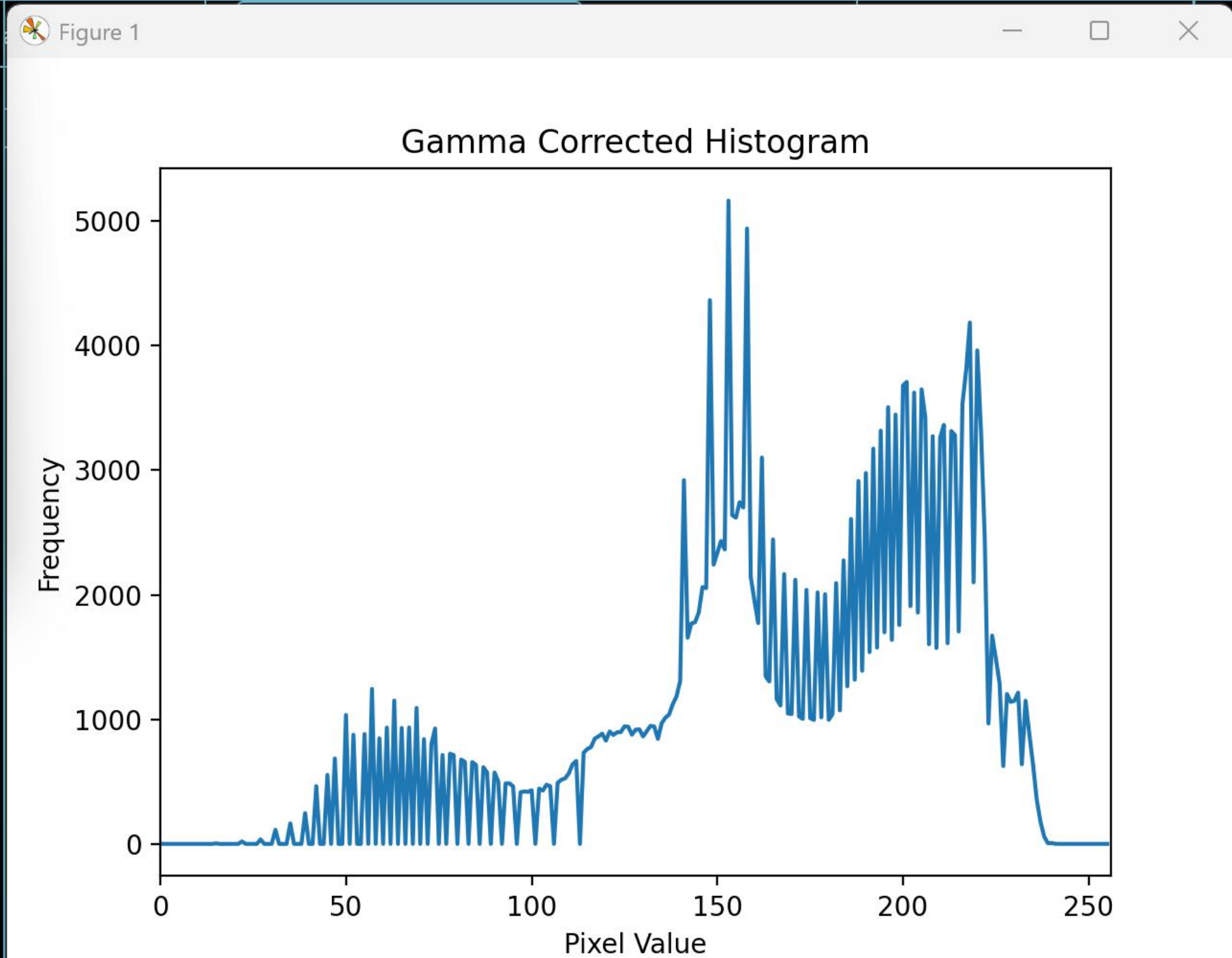
```
76      plot_histogram(
77
78      cv2.imshow("Neg
79      plot_histogram(
80
81      # Wait and clos
82      cv2.waitKey(0)
83      cv2.destroyAllW
```

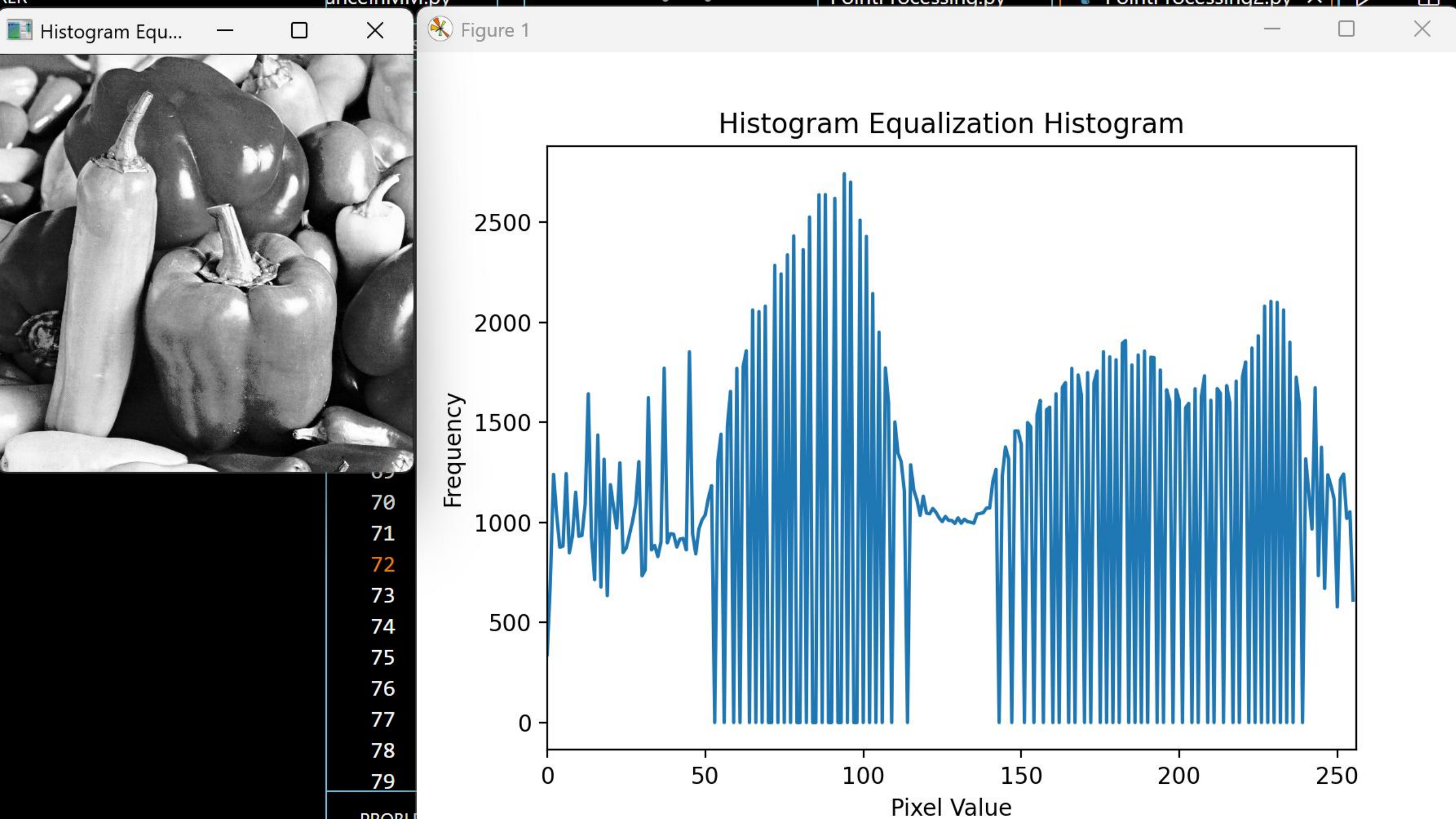
PROBLEMS OUTPUT DEBUG CO

Figure 1

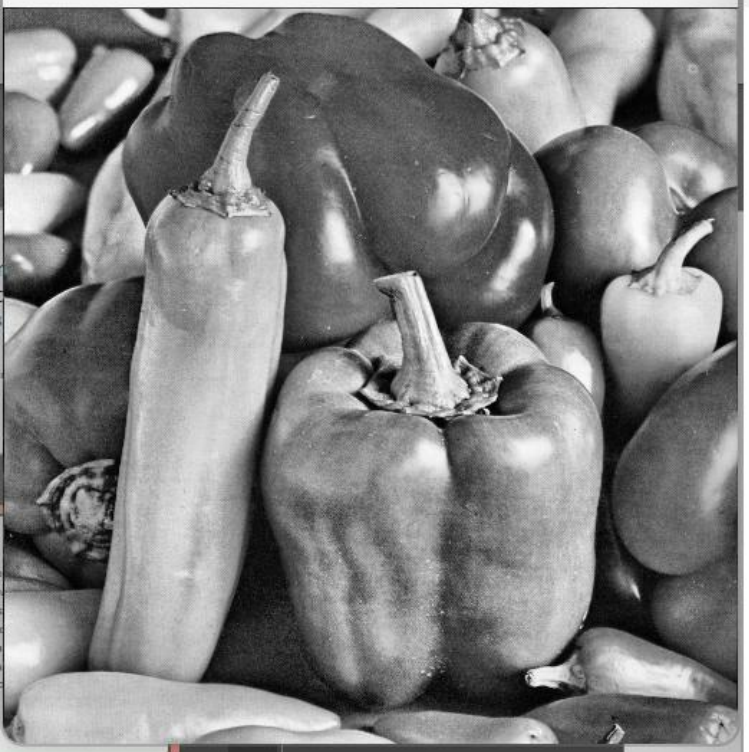



Gamma Correc...


Figure 1
Gamma Corrected Histogram
Frequency
5000
4000
3000
2000
1000
0
0
50
100
150
200
250
Pixel Value

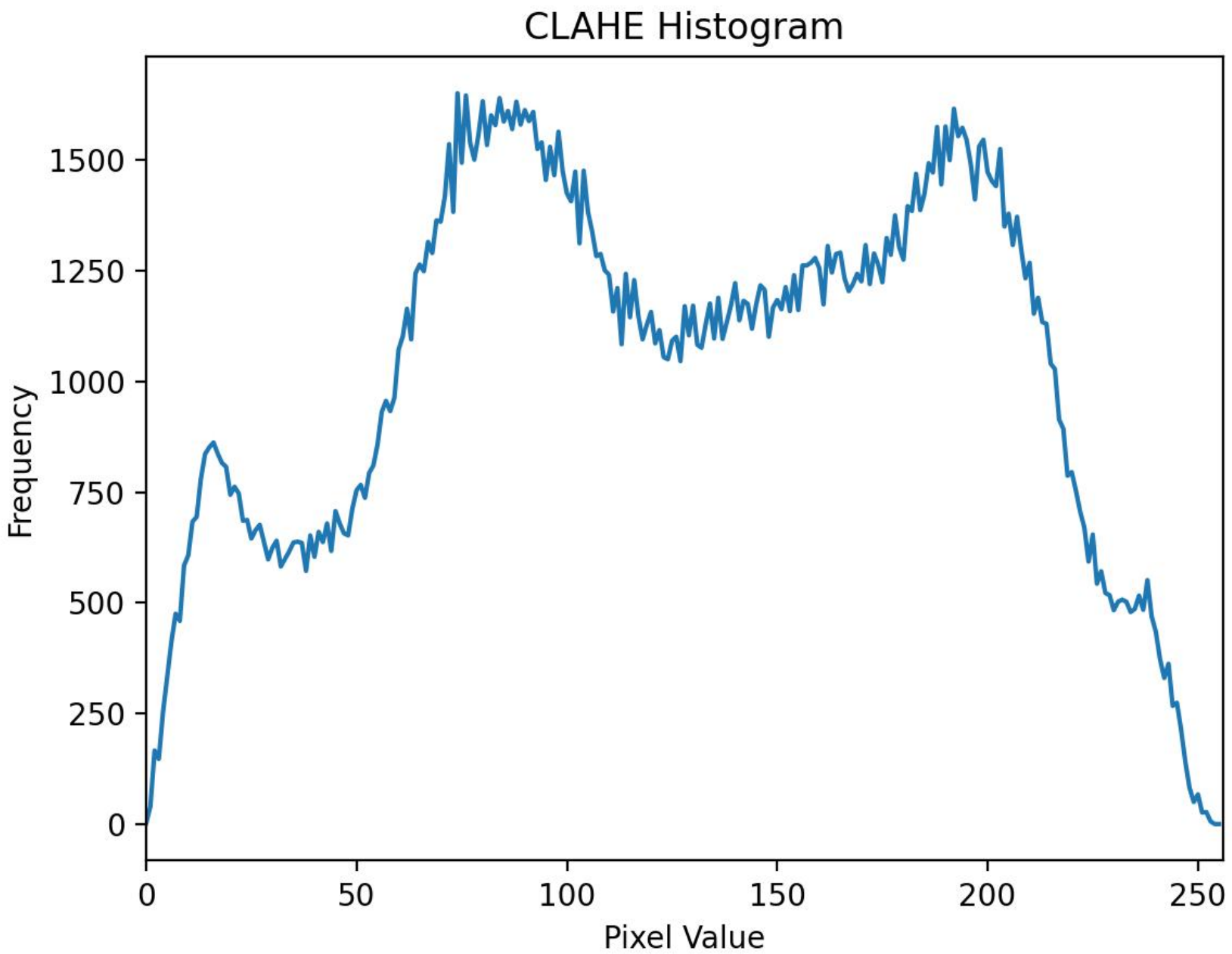

Histogram Equ...
Figure 1
Histogram Equalization Histogram
Frequency
Pixel Value
0
500
1000
1500
2000
2500
0
50
100
150
200
250
70
71
72
73
74
75
76
77
78
79

CLAHE Image



nt Proces...

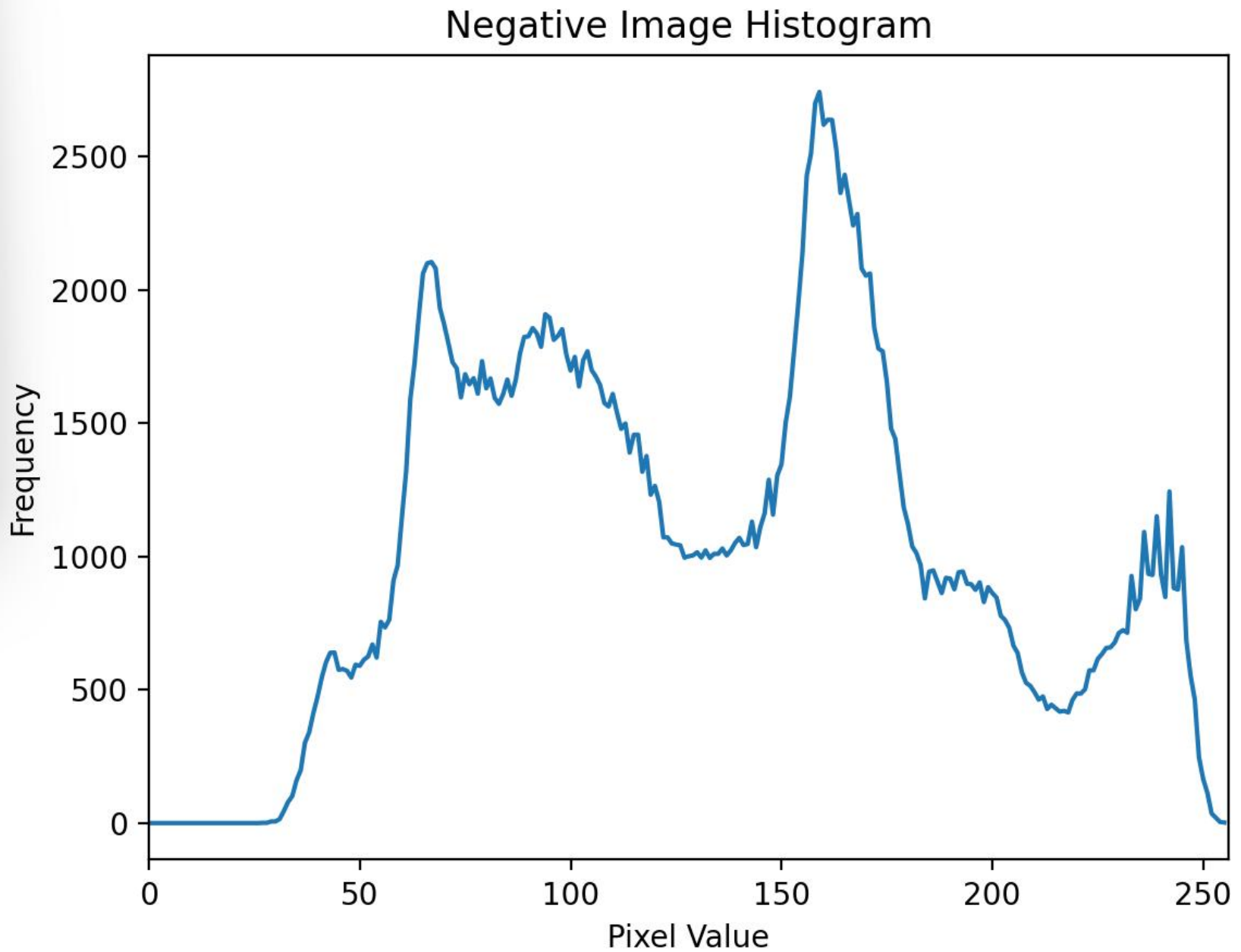
Figure 1



Negative Image



Figure 1



70
71
72
73
74
75
76
77
78
79

PROBLEMS

Task 2

- Load an image and convert it to grayscale.
- Generate **five noisy versions** of the same image (using Gaussian noise).
- **Average** those five noisy images.
- Compare the result with the **original** grayscale image.

Welcome | MultiImageAvrage.py

MultiImageAvrage.py > add_gaussian_noise

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def add_gaussian_noise(image, mean=0, std=40):
    """
    Add Gaussian noise to a grayscale image.

    :param image: Input grayscale image as a NumPy array (float32 or float64).
    :param mean: Mean of the Gaussian noise.
    :param std: Standard deviation of the Gaussian noise.
    :return: Noisy image with the same dimensions as the input, range clipped to [0, 255].
    """
    noise = np.random.normal(mean, std, image.shape)
    noisy_image = image + noise
    # Clip to ensure valid range for display
    noisy_image = np.clip(noisy_image, 0, 255)
    return noisy_image

def main():
    # 1. Load an image in colour, then convert to grayscale
    img_color = cv2.imread("images/Peper.tiff")  # Replace image with your image path
    if img_color is None:
        raise FileNotFoundError("Image not found. Please check the file path.")

    # Convert to grayscale and float32
    img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY).astype(np.float32)

    # 2. Create 5 noisy versions of the grayscale image
    noisy_images = []
    for i in range(5):
        noisy = add_gaussian_noise(img_gray, mean=0, std=15)  # Slightly higher std for mild noise
        noisy_images.append(noisy)
```

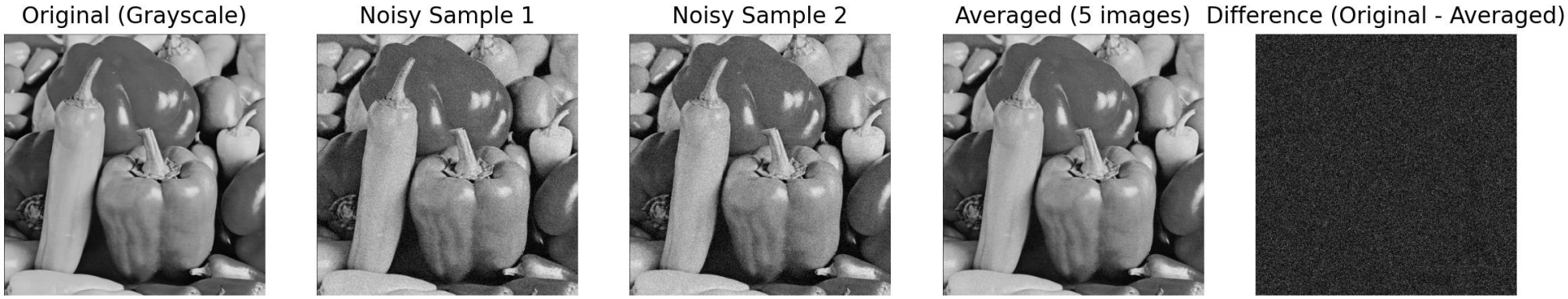
```
34
35      # 3. Display two samples from the 5 noisy images (for instance, 1st and 3rd)
36      sample_noisy_1 = noisy_images[0]
37      sample_noisy_2 = noisy_images[2]
38
39      # 4. Average the 5 noisy images
40      sum_noisy = np.zeros_like(img_gray, dtype=np.float32)
41      for noisy_img in noisy_images:
42          sum_noisy += noisy_img
43      avg_image = sum_noisy / 5.0  # Element-wise average
44
45      # 5. Compare the averaged image with the original by subtracting one from the other
46      #    We'll take the absolute difference for a positive-valued difference map
47      difference = np.abs(img_gray - avg_image)
48
49      # For visualisation, we can normalise the difference to [0, 255] so it's easier to see
50      diff_min, diff_max = difference.min(), difference.max()
51      diff_display = ((difference - diff_min) / (diff_max - diff_min + 1e-8)) * 255.0
52
53      # 6. Display all relevant images using matplotlib
54      fig, axs = plt.subplots(1, 5, figsize=(20, 4))
55
56      axs[0].imshow(img_gray, cmap="gray", vmin=0, vmax=255)
57      axs[0].set_title("Original (Grayscale)")
58      axs[0].axis("off")
59
60      axs[1].imshow(sample_noisy_1, cmap="gray", vmin=0, vmax=255)
61      axs[1].set_title("Noisy Sample 1")
62      axs[1].axis("off")
```

```
63
64      axs[2].imshow(sample_noisy_2, cmap="gray", vmin=0, vmax=255)
65      axs[2].set_title("Noisy Sample 2")
66      axs[2].axis("off")
67
68      axs[3].imshow(avg_image, cmap="gray", vmin=0, vmax=255)
69      axs[3].set_title("Averaged (5 images)")
70      axs[3].axis("off")
71
72      axs[4].imshow(diff_display, cmap="gray", vmin=0, vmax=255)
73      axs[4].set_title("Difference (Original - Averaged)")
74      axs[4].axis("off")
75
76      plt.show()
77
78  if __name__ == "__main__":
79      main()
80
```

Code explanation

- **add_gaussian_noise**
  - Increases the standard deviation (std=15) slightly to introduce a bit more noise (but still moderate).
  - Clamps (np.clip) pixel values to [0, 255] to avoid out-of-range data.
- **Displaying Two Samples**
  - We select the **first** and **third** noisy images (sample_noisy_1, sample_noisy_2) to showcase variety.
- **Averaging and Difference**
  - We **sum** all five noisy images and **divide** by 5 to get avg_image.
  - We compute difference = |img_gray - avg_image| to see how the averaged image differs from the original pixel by pixel.
- **Normalising the Difference**
  - The difference map could be small or large, so we **normalise** it to [0, 255] for better visibility when displaying.
- **Visualisation**
  - We create a row of five subplots: **Original**, **Noisy Sample 1**, **Noisy Sample 2**, **Averaged**, and **Difference**.
  - This layout helps us quickly compare outcomes and see how averaging reduces noise.

Figure 1


Original (Grayscale)
Noisy Sample 1
Noisy Sample 2
Averaged (5 images)
Difference (Original - Averaged)

Optional Assignment

- You have previously explored various image enhancement techniques. In this assignment, you will focus on a Retinex-based enhancement method, which aims to improve image visibility by modelling how the human visual system perceives colour and illumination.

Optional Assignment

1. **Theory**
    1. **Retinex Concept**:
        1. Explain the underlying principles of Single-Scale Retinex or Multi-Scale Retinex.
        2. Discuss how this approach accounts for varying lighting conditions to produce more natural-looking images.
    2. **Human Perception**:
        1. Briefly illustrate how Retinex theory relates to the way human eyes adjust and interpret scenes with uneven or harsh lighting.
2. **Implementation**
    1. **Algorithm**:
        1. Implement a **basic Retinex algorithm** in Python (e.g., using OpenCV or NumPy).
    2. **Testing**:
        1. Apply your Retinex implementation to images with challenging lighting conditions (e.g., low-light or high-contrast scenes).
        2. Show original vs. enhanced results.
3. **Results and Discussion**
    1. **Visual Comparison**:
        1. Include before-and-after images demonstrating how Retinex improves visibility.
    2. **Strengths and Limitations**:
        1. Briefly discuss the algorithm's advantages (e.g., enhanced detail, more natural colour) and any noticeable drawbacks (e.g., potential artefacts).

Create a PowerPoint presentation that includes the results of your assignment, the theory behind each task, and the relevant Python code. Then email it to j.dehmeshki@kingston.ac.uk

Completing this task can earn you up to 3 additional marks on your coursework.